Estruturas de Dados

Trabalho Vivencial.

Professor: Prof. Msc. Paulo Cirillo Souza Barbosa

Introdução.

O presente trabalho trata-se da implementação de uma versão extendida do processo de autenticação textual. Tal procedimento, é realizado por meio de um algoritmos em que se garante a identificação correta dos autores em um documento expedido de modo eletrônico. Essa ferramenta consiste em um mecanismo capaz de assegurar a veracidade da identidade do signatário de um documento, o que é fundamental para proporcionar a segurança jurídica em procedimentos legais de diferentes naturezas, como petições, transações comerciais e acordos. Além de garantir que o arquivo foi gerado por uma pessoa autorizada e devidamente identificada, a autenticação digital também confirma a origem e a integridade de determinado documento. Dessa forma, é possível visualizar se houve alguma alteração no documento eletrônico, por menor que ela tenha sido, o que o resguarda de eventuais interceptações que comprometam o seu conteúdo. Por fim, a autenticação digital é ainda uma ferramenta útil para autorizar os leitores finais de um documento, já que só terão acesso ao arquivo após passarem por um processo de confirmação de identidade. Essa característica é muito útil já que permite não apenas assegurar a veracidade do emissor do documento, mas também controlar quem serão os destinatários do arquivo enviado, mantendo o sigilo das informações a outras pessoas.

Descrição do Projeto

O projeto a ser desenvolvido é uma versão de autenticador digital, ou seja, um aplicativo para verificação da autenticidade de documentos. O aplicativo deverá gerar códigos de autenticação para documentos textuais. A modificação do algoritmo é feita como um exercício que usa as implementações das principais estruturas de dados implementadas ao longo do curso. Assim, a geração do código de autenticação do documento deverá ser feita com o auxílio de um conjunto de estruturas de dados. Cada estrutura tem um papel específico na geração de tal código e segue a seguinte sequência de passos:

1. Realizar a leitura de um arquivo textual, em arquivo .TXT. **Obs:** A equipe está livre para produzir um parágrafo com um texto que seja em português-BR que possua no mínimo 30 linhas;
2. Cada linha do texto é consumida individualmente e as palavras presentes nessa linha textual são inseridas em uma lista **dinâmica**. Nesse caso, as palavras são obtidas ao identificar os espaços em branco entre textos.
3. Deve-se em seguida, acessar os elementos da lista no sentido reverso (do final até o início) e inserir tais elementos um a um em uma árvore AVL. Para realizar as comparações lexicográficas Strings, pode-se utilizar o método **compareToIgnoreCase()**. Além disso, deve-se desconsiderar quaisquer palavras duplicadas na String em questão.
4. Após o cadastro de todas as Strings para aquela linha, deve-se inserir a árvore AVL em uma pilha.
5. Os procedimentos devem ser repetidos para todas as linhas no texto.
6. Após a pilha de árvores ser construída, deve-se desempilhar as árvores de uma em uma, e gerar o código hash para aquela árvore desempilhada.
7. Cada hash gerado de cada uma das árvores deve ser concatenado e separados por uma quebra de linha.

O fluxo de construção da pilha de árvores pode ser visualizado na Figura 1.

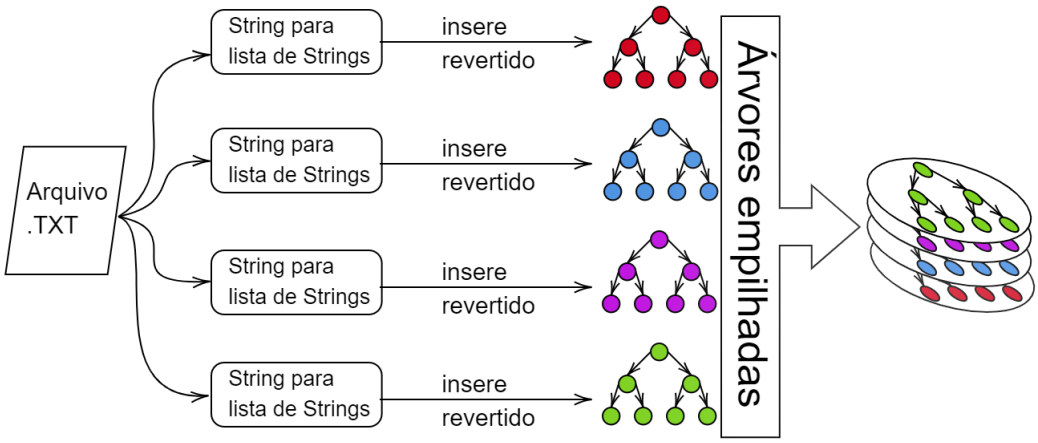


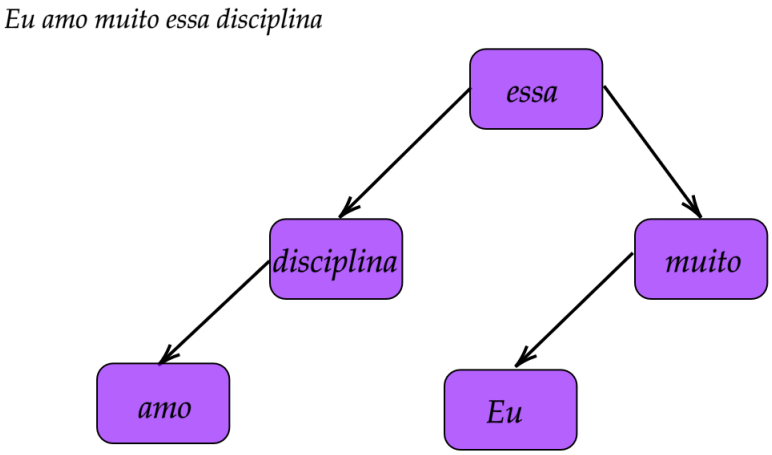
Figura 1: Fluxograma de construção da pilha de árvores

Para a construção do hash para uma única linha textual presente em um arquivo, considere o exemplo em a linha lida tem a seguinte String: "*Eu amo muito essa disciplina!*". A sequência de passos para construção da árvore é a seguinte:

1. Divida a String em uma lista de substrings separadas por espaço, obtendo: ["Eu", "amo","muito", "essa", "disciplina!"]

2. Em seguida, faça a inserção dos itens na árvore balanceada no sentido reverso (do fim até o início, ou seja, de "disciplina"até "Eu") de que foram inseridos na lista. Para o balanceamento, considere o uso da análise lexicográfica das Strings por meio do método **compareToIgnoreCase()**.

Uma observação **importante** é que como se trata de uma árvore binária, strings duplicatas não deverão ser inseridas na estrutura, neste caso quando o método **compareToIgnoreCase()** retornar 0, as Strings são iguais. Segue exemplo da árvore construída após a inserção da String exemplo:



Obs: Note que esta árvore já se encontra balanceada.

Como passo final, após a inserção dos elementos na árvore, deve-se realizar a travessia dos nós e computar o *hash* da árvore através da lógica: os nós pais armazenam o *hash* da combinação dos *hash* de seus filhos. Por exemplo, se A e B são filhos de um nó P, então o *hash* de P será $h(h(A) + h(B) + h(P))$. Assim, o hash computado para aquela String, encontra-se armazenado no nó raiz.

Assim, o hash final encontra-se no nó raíz da árvore AVL e para este exemplo específico, pode ser visualizado na figura 2:

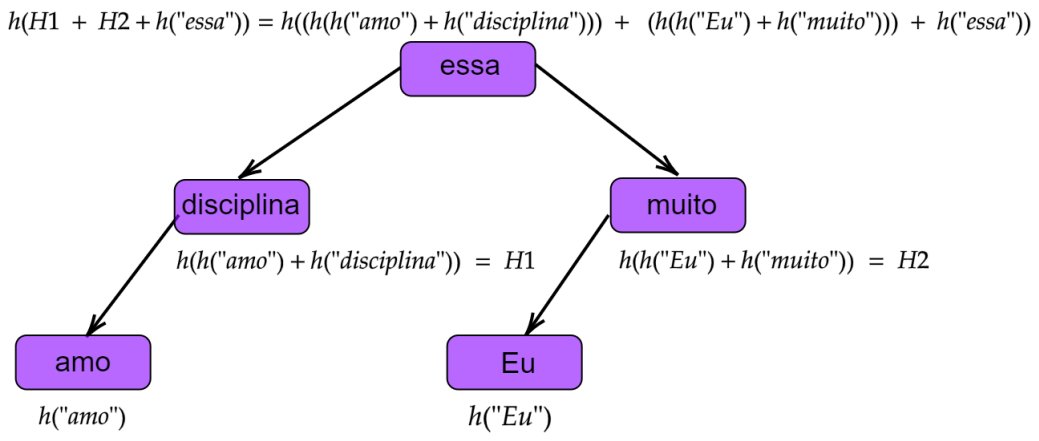


Figura 2: Exemplo geração de hashes

Destaca-se que quando não há filhos a esquerda ou direita, não se deve aplicar um hash para *null*. Para esse caso específico, o hash final gerado pelo nó raiz é: *f44040deec0de6e3100f7ff772bb7a25646e71ed*

Novamente, é importante destacar que o processo em questão, é operado em apenas uma linha textual. Desta maneira, caso seja de desejo computar o autenticador para um texto com múltiplas linhas, necessário realizar o procedimento descrito para cada linha.

Informações Adicionais

Os requisitos do projeto são:

1. Deve-se receber como entrada, um arquivo de texto (formato .TXT).
2. Gerar como saída o código de autenticação do documento.
3. A geração do código de autenticação do documento deverá ser realizada conforme procedimento descrito na seção 1.
4. Para cada String (linha) presente no texto, deve-se criar uma árvore com a estrutura descrita na seção anterior.
5. Deve-se armazenar cada uma dessas árvores em uma pilha e gerar o código de cada linha de texto, ao desempilhar cada árvore e realizar a travessia computando cada **hash**. O código gerado para cada árvore deve ser impresso separado por quebra de linha.
6. Os *hashes* deverão ser gerados pela função de dispersão criptográfica SHA-1. Um gerador online e o código-fonte da função pode ser encontrado no site SHA-1 online.

Critérios Avaliativos.

- A equipe deve realizar um envio de uma pasta compactada contendo:
  1. As implementações realizadas no trabalho. A pontuação deste é (0 até 1,25).
  2. Documento relatório Que possui pontuação de 0 até 2,5. Este deve ser escrito utilizando o modelo de artigo científico disponibilizado neste **LINK**. A avaliação do relatório será realizada da seguinte forma (descrita em proporção):
     - Título (5%).
     - Metodologia (47,5%).
     - Resultados (47,5%).

3. Além das implementações, sua equipe deve gravar um vídeo de no máximo 10 minutos, no qual cada membro da equipe deve ter um papel protagonista na explicação do trabalho. Assim, é **obrigatório** que no início de fala de cada membro seja dito qual nome e matrícula. A pontuação também será de (0 até 1,25), porém, esta será condicionada a apresentação realizada pelo aluno.

6) Observações.

- Obs1: O trabalho é de equipe composta de 6 alunos no máximo.
- Obs2: A data estipulada para entrega do trabalho, também é um critério avaliativo. Assim, a nota da equipe será **zero** caso o envio não seja realizado.
- Ob3: Os relatórios e implementações serão enviadas a um software anti-plágio. Qualquer caracterização de plágio ocasionará em nota zero para ambas equipes.